SystemVerilog Assertions

习题集与面试题库（50题精华版）

覆盖范围：基本语法 · 序列与属性 · 时序操作符 · 采样语义 · 覆盖率 · 形式验证

难度分级：初级 / 中级 / 高级　参考标准：IEEE Std 1800-2023

第一章：基本语法与断言类型（Q1–Q10）

Q1 【初级】 立即断言与并发断言的差别是什么？请写出各一条示例断言。

▶ 答案

立即断言（immediate assertion）在过程代码执行到该语句时，对一个非时序表达式进行判断，语义类似 if 条件；并发断言（concurrent assertion）基于时钟采样，在每个采样 tick 触发新的评估尝试（attempt），对跨多个周期的序列/属性进行检查。

工程经验：捕获周期内毛刺/组合瞬态 → 立即断言；验证协议/握手/管线延迟等跨周期行为 → 并发断言。

▶ 示例代码

```
// 立即断言：过程执行时立即检查（组合逻辑/时刻相关）

always_comb begin

  assert (!(a && b)) else $error("a and b cannot both be 1");

end


// 并发断言：在 clk 的采样边沿检查跨周期关系

property p_req_ack_1cycle;

  @(posedge clk) disable iff (!rst_n) req |-> ##1 ack;

endproperty

ap_req_ack_1cycle: assert property (p_req_ack_1cycle)

  else $error("ack must be high 1 cycle after req");
```

▶ 注意事项

陷阱：漏写 |-> 直接写 req ##1 ack，每个周期都尝试匹配，req=0 的周期也会失败（经典 GOTCHA）。

Q2 【中级】 deferred immediate assertions：`assert #0` 与 `assert final` 有什么区别？什么时候用？

▶ 答案

deferred immediate 仍是立即断言：表达式在语句被处理时求值，但报告/动作被延后执行，以降低组合毛刺造成的误报。标准给出两类：

- assert #0（observed deferred）：动作延后到当前 time step 的 Observed region。
- assert final（final deferred）：动作延后到当前 time step 的 Final region，更晚执行。

action block 通常只能包含单个子程序调用（如单个 $error(...)），不能写多语句块。

▶ 示例代码

```
// 简单立即断言：可能对瞬态/毛刺敏感

always_comb begin

  assert (!(a && c)) else $error("IMMEDIATE: a&&c glitch or violation");

end


// deferred immediate：表达式同时求值，但动作延后

a_obs:  assert #0   (!(a && c)) $error("DEFERRED #0: a&&c observed");

a_fin:  assert final(!(a && c)) $error("DEFERRED final: a&&c at final");
```

▶ 注意事项

常见误区：以为 deferred 会重新采样表达式，实际上表达式仍在语句处理时求值，区别仅在于动作调度到更晚区域。

Q3 【初级】 `assert/assume/cover property`各自的语义与用途是什么?

▶ 答案

- assert property：断言该属性必须成立，违反时报错。
- assume property：把属性当作环境假设，形式验证中约束输入空间；仿真中也可检查假设是否被违反。
- cover property：记录某个行为是否发生（覆盖点），统计 attempted/succeeded 次数。

工程技巧：同一 property 同时 assert + cover，前者保证正确性，后者证明测试确实触发了该场景（避免 vacuous）。

▶ 示例代码

```
property p_req_to_ack;

  @(posedge clk) disable iff (!rst_n) req |-> ##[1:3] ack;

endproperty


ap: assert property (p_req_to_ack) else $error("REQ->ACK violated");

as: assume property (p_req_to_ack) else $error("Assumption violated (env bug)");

cp: cover  property (p_req_to_ack);
```

▶ 注意事项

陷阱：在仿真里把 assume property 当成不会报错的约束是错的，标准允许默认调用 $error。

Q4 【中级】 `expect` 语句是什么？与 `assert property` 有何根本区别？

▶ 答案

> expect (property_spec) action_block 是过程阻塞语句：执行到 expect 后，当前进程阻塞，直到该 property 成功或失败，然后执行 pass/else 动作并继续向下运行；它只启动单个线程的 property 评估，完成后停止。
> 而 assert property（静态形式）从仿真开始到结束持续在每个时钟 tick 启动新的 attempt。
> expect 的第一次评估从下一次 clocking event 开始，不是在语句执行的同一 tick。

▶ 示例代码

```
program automatic q4_expect_prog (input logic clk, input logic a, b, c);

  initial begin

   // 等待一次性事件序列 a ##1 b ##1 c

    expect ( @(posedge clk) a ##1 b ##1 c )

      $info("expect succeeded");

    else

      $error("expect failed");

    $info("program continues after expect completes");

  end

endprogram
```

▶ 注意事项

> 常用于 testbench 中等待某个协议阶段结束再推进下一步，例如等待初始化序列完成。

Q5 【中级】 action block 能写什么？默认失败动作是什么？并发断言的 pass/fail 在哪个事件区执行？

▶ 答案

> action block 用于声明断言成功/失败时执行的动作（$info/$error/$fatal 等）。若并发断言省略 else，标准规定默认调用 $error。
> 并发断言的 pass/fail 语句在 Reactive region 执行。
> action block 中不得再包含并发 assert/assume/cover 语句，以免形成递归/不确定调度。

▶ 示例代码

```
property p;

  @(posedge clk) disable iff (!rst_n) req |-> ##1 ack;

endproperty
```

```
ap: assert property (p)
      $info("PASS: req->ack ok");
    else
      $error("FAIL: req->ack violated");
```

▶ **注意事项**

> 多语句动作应封装成 task/function，在 action block 中只调用一次。

Q6 【高级】 如何使用 `$assertcontrol`（以及 `$asserton/$assertoff/$assertkill`）控制断言开关与 pass/fail 动作?

▶ **答案**

> 标准给出 $assertcontrol(control_type [, assertion_type] [, directive_type] [, levels] [, list...])，以及简写任务 $asserton/$assertoff/$assertkill 等。核心区别：
>
> - Off（control_type=4）：停止新 attempt 启动，不影响已执行的 attempt。
> - Kill（control_type=5）：立即中止当前 attempt 并 flush pending 实例。
> - 还可控制 PassOff/FailOff、NonvacuousOn（只在 nonvacuous success 时执行 pass）等。

▶ **示例代码**

```
initial begin
  // 关闭所有断言（Off = control_type 4）
  $assertcontrol(4);
  // 等复位释放后开启（On = control_type 3）
  wait (rst_n === 1'b1);
  $assertcontrol(3);
  // 只在 nonvacuous success 时执行 PASS 动作
  // assertion_type=1(Concurrent), directive_type=1(Assert)
  $assertcontrol(10, 1, 1);
end
```

▶ **注意事项**

> 陷阱：$assertoff 不会立刻清空正在跑的线程；需要立即终止并清空队列，应使用 $assertkill（Kill 类型）。

Q7 【中级】 什么是 vacuous success？为什么它是"看似全绿但其实没验证"的根源？

▶ 答案

标准定义：一次 property evaluation attempt 要么是 vacuous、要么是 nonvacuous。典型例子：a |-> b 在 a 为假时会得到 vacuous success，此时结果与 b 无关。

长期只有 vacuous success 可能意味着设计或属性表述存在问题，不应被当作验证通过的信号。

工程上用 cover property 或 nonvacuous 控制来确认 antecedent 真的发生过。

▶ 示例代码

```
property p_a_implies_b;

  @(posedge clk) disable iff(!rst_n) a |-> b;

endproperty


ap: assert property(p_a_implies_b) else $error("a implies b violated");

// 统计成功/尝试/因 vacuity 成功等

cp: cover  property(p_a_implies_b);

// 单独覆盖 antecedent 是否触发

c_a: cover property(@(posedge clk) a);
```

▶ 注意事项

标准在 cover statement 中单列"因 vacuity 成功次数"统计字段，帮助识别此问题。

Q8 【中级】 为什么在 `always_comb` 里写立即断言可能出现"毛刺误报"或"同一时刻重复报错"？如何缓解？

▶ 答案

always_comb 的隐式敏感列表会把过程里读取到的信号都加入敏感集合，包括断言表达式中的信号（但 action block 不参与）。因此组合输入多次变化会导致立即断言多次执行；若组合逻辑存在周期内瞬态，会触发误报。

缓解方案：

1. 使用 deferred immediate（assert #0 / assert final）延后动作；
2. 改用并发断言在时钟边沿采样（适合同步设计）。

▶ 示例代码

```
// 组合过程：输入变化会反复触发

always_comb begin

  assert (!(a && b && c)) else $error("immediate fired (maybe transient)");

end
```

```
// deferred：更适合避免同一 time step 的瞬态误报
assert #0 (!(a && b && c)) $error("deferred #0 fired");
```

▶ 注意事项

同步设计建议用并发断言只在边沿采样，更契合寄存器只在边沿锁存的行为。

Q9 【中级】 断言应该写在 RTL 里（embedded）还是用 bind 单独挂载（bind-in）？请给出工程化取舍与一个最小示例。

▶ 答案

工程分工建议：

- RTL 工程师：在 RTL 中嵌入输入合法性、状态编码、关键不变量等断言。
- 验证工程师：用 bind 添加规格级协议检查、corner case、覆盖点，不改 RTL 文件也不触发综合流程。

关键限制：通常只有并发断言适合通过 bind 挂到其他模块/接口中。

对 IP 多实例：用 bind <module_name> 给所有实例统一挂载；单实例用 bind <hier_path>。

▶ 示例代码

```
module dut_sva(input logic clk, rst_n, req, ack);
  property p;
    @(posedge clk) disable iff(!rst_n) req |-> ##1 ack;
  endproperty
  ap: assert property(p) else $error("bind-in: req->ack violated");
endmodule

// bind：不修改 dut 源文件即可插入检查
bind dut dut_sva u_dut_sva(.clk(clk), .rst_n(rst_n), .req(req), .ack(ack));
```

▶ 注意事项

断言离被检查 RTL 太远会降低可读性，建议通过命名规范、分层 bind 文件、就近封装成 checker 缓解。

Q10 【初级】 `$onehot/$onehot0/$isunknown` 如何用于状态机与输入合法性检查？与"自定义 onehot"相比有何坑?

▶ 答案

- $onehot(expr)：检查 expr 中为 1 的 bit 数是否恰好为 1（对 X/Z 有特殊处理）。
- $onehot0(expr)：允许 0 或 1 个 bit 为 1。
- $isunknown(expr)：检测 expr 中是否含有 X/Z。

> 坑：$onehot 对 4 值逻辑中的 X/Z 处理不等同于"严格 onehot"。若要求"任何 X/Z 都算失败"，需显式组合 $isunknown 做硬约束。

▶ 示例代码

```
always_comb begin

  if (init_done) begin

    assert ($onehot(state))     else $error("state is not onehot");

    assert (!$isunknown(state)) else $error("state has X/Z");

  end else begin

   // 初始化期间允许全 0（onehot0 作为 reset state）

    assert ($onehot0(state)) else $error("state not onehot0 during init");

  end

end
```

▶ 注意事项

> 对控制输入互斥检查（如 start/stop 不能同时为 1），可用 $onehot 简洁表达。

第二章：序列与属性建模（Q11–Q20）

Q11 【初级】 `sequence` 与 `property` 有何区别？为什么常说"sequence 是 building block，property 是可断言的规范"？

▶ 答案

> sequence 描述跨周期的事件序列（sequence_expr）；property 描述可判定真/假的属性表达式（property_expr），允许用 implication 等构造把序列与属性组合成条件化规则。
> 工程实践：把复杂协议切成多个命名 sequence，再在 property 里组合，便于复用与分层。

▶ 示例代码

```
sequence s_req;

  req;

endsequence


sequence s_ack_after_2;

  ##2 ack;
```

```
endsequence


property p_req_ack_when_ready;

  @(posedge clk) disable iff(!rst_n)

    (ready && s_req) |-> s_ack_after_2;

endproperty


ap: assert property (p_req_ack_when_ready) else $error("req->ack rule broken");
```

▶ 注意事项

> 把"纯序列"直接塞进 assert property 是允许的，但要保证该序列在 property 语境下合法。

⚠ 勘误说明

> 原文 Q11 代码中使用了 (ready and s_req)，此处 and 是序列并行操作符。由于 ready 是布尔信号而非序列，应使用布尔操作符 &&，改为 (ready && s_req) 语义更规范，避免歧义。

Q12 【中级】 如何写"参数化"的 sequence/property？尤其是把 clocking event 作为参数传入的写法？

▶ 答案

> 标准允许 sequence/property 带形式参数，包括整型范围、信号，甚至 event 类型参数。可将 delay、repeat 范围参数化，也可把整个 clocking event 作为 event 参数传入。
> 注意：event 参数是传递整个 clocking event（如 posedge clk），不是传入一个信号再与 posedge 组合。

▶ 示例代码

```
// 参数化延迟与重复

sequence s_delay_repeat (int d1, int min_rep, int max_rep, int d2);

  x ##d1 y[*min_rep:max_rep] ##d2 z;

endsequence


// event 参数化：把时钟事件作为参数

sequence s_on_event (event ev);

  @(ev) x ##1 y;

endsequence


c1: cover property (s_delay_repeat(2, 1, 3, 1));
```

```
c2: cover property (s_on_event(posedge clk));
```

▶ 注意事项

用参数化断言封装协议（max_latency、min_hold、timeout），在 SoC 多实例中复用同一套断言。

Q13 【高级】 序列/属性中的局部变量怎么用？什么是 local-variable flow，为什么它是高级坑点？

▶ 答案

SVA 支持在 sequence/property 中声明局部变量，并在匹配过程中赋值。局部变量可见性与流出（flow out）有严格规则：

- 局部变量在声明体内可见，外部实例化上下文默认不可直接访问。
- 若需要把值带出，通常把外部变量作为 untyped 参数传入并在内部赋值。

并行组合（and/or/intersect）会产生多线程匹配，局部变量在多线程中被写入时外部看到的值可能不一致。

▶ 示例代码

```
// 错误示例：sub_seq1 的局部变量 v1 在外部不可见

sequence sub_seq1;

  int v1;

  @(posedge clk) (a ##1 !a, v1 = data_in) ##1 b && (data_out == v1);

endsequence


// 正确姿势：把外部变量 v_capture 作为参数传入，内部赋值后外部可用

sequence sub_seq2(untyped lv);

  @(posedge clk) (a ##1 !a, lv = data_in) ##1 b && (data_out == lv);

endsequence


sequence top_seq;

  int v_capture;

  sub_seq2(v_capture) ##1 (v_capture != 0);

endsequence


c1: cover property (top_seq);
```

▶ 注意事项

常用于捕获事务 payload（地址/数据/ID），并在后续周期与返回值对比。

Q14 【中级】 `|->` 与 `|=>` 的精确定义是什么？"空匹配（empty match）"对 `|=>` 有何影响？

▶ 答案

- |->（overlapped implication）：consequent 从 antecedent 匹配的 end point 开始评估（同 tick）。
- |=>（nonoverlapped implication）：consequent 从 antecedent 非空匹配的 end point 的下一个 clock tick 开始。

等价关系：seq |=> p 等价于 seq ##1 p（非重叠仅是重叠加一拍延迟的语法糖）。

两者共同点：若从某 start point 起 antecedent 没有任何匹配，则该 start point 的 implication 评估 vacuously 成功。

▶ 示例代码

```
// overlapped: req 为真时，同一 tick 要求 ack_same 为真

ap_ovl: assert property (@(posedge clk) disable iff(!rst_n) req |-> ack_same)

  else $error("overlapped implication failed");


// nonoverlapped: req 为真时，下一个 tick 要求 ack_next 为真

// 等价于: req |-> ##1 ack_next

ap_non: assert property (@(posedge clk) disable iff(!rst_n) req |=> ack_next)

  else $error("nonoverlapped implication failed");
```

▶ 注意事项

把需要条件触发的规则写成无 implication 的纯序列会导致 req 不发生的周期也失败。

Q15 【高级】 `and/or/intersect` 在序列组合上的语义差别是什么？为什么说 `intersect` 比 `and` 更"严格"？

▶ 答案

- and：两个操作数序列都必须匹配，但两段匹配长度可以不同。
- or：任意一个操作数序列匹配即可。
- intersect：两个操作数都必须匹配，且两段匹配长度必须相同——这是 intersect 相对 and 的核心区别。

intersect 适合表达"两个序列在同一时间窗对齐地发生"。

▶ 示例代码

```
sequence s1; @(posedge clk) a ##1 b; endsequence

sequence s2; @(posedge clk) c ##1 d; endsequence
```

```
// intersect: 要求 s1 与 s2 的匹配区间长度一致
ap_int: assert property (@(posedge clk) (s1 intersect s2))
  else $error("intersect failed");

// or: 任意一个序列匹配即可
ap_or:  assert property (@(posedge clk) (s1 or s2))
  else $error("or failed");
```

▶ **注意事项**

> intersect/within/throughout 等运算符与 ##/重复结合容易出优先级问题，建议强制加括号。

Q16 【高级】 `first_match` 解决了什么问题？什么时候不用它反而更好？

▶ **答案**

> sequence 候选匹配可能产生多个并行 threads；first_match(seq) 选择"最早结束的匹配"。这可以避免某些 antecedent 因"永远有更长的匹配在路上"而导致 assertion 线程无法到达成功终点。
> 不适合用 first_match 的情况：若需求需要覆盖所有匹配（例如每次 b 都要触发后续检查），盲目 first_match 会漏检。

▶ **示例代码**

```
sequence s;
  $rose(a) ##[2:$] b; // 区间可能产生多线程匹配
endsequence

// 使用 first_match 限制只取最早的 b 匹配点
ap_fm: assert property (@(posedge clk) disable iff(!rst_n)
          first_match(s) |=> c)
        else $error("c not after first match of s");
```

▶ **注意事项**

> 把 ##[2:$] 换成有限上界可减少线程爆炸；或用 ##[m:n] 明确窗口后不需要 first_match。

Q17 【中级】 `throughout` 的语义是什么？请写一个"burst_mode 低电平必须贯穿整个子序列"的序列。

▶ **答案**

> exp throughout seq 的等价形式为 (exp)[*0:$] intersect seq，语义是：在 seq 匹配的整个有限区间内，exp 每个 tick 都必须为真。
> 常用于：握手完成前信号必须保持稳定、禁用标志在整个传输窗口生效等。

▶ 示例代码

```
sequence s_sub;

  ##2 ((trdy==0) && (irdy==0))[*7];

endsequence


// burst_mode 必须在 s_sub 覆盖的整个区间内保持为 0

sequence s_burst_rule;

  @(posedge mclk) $fell(burst_mode) ##0 ((!burst_mode) throughout s_sub);

endsequence


ap: assert property (s_burst_rule) else $error("throughout failed");
```

▶ 注意事项

> 陷阱：throughout 约束的是 seq 的匹配区间，seq 长度可变时可能导致约束范围与预期不符。

Q18 【中级】 `within` 的语义是什么？如何表达"子序列必须发生在大序列内部的某个子区间"？

▶ 答案

> seq1 within seq2 的等价缩写：(1[*0:$] ##1 seq1 ##1 1[*0:$]) intersect seq2。
> 直觉上：seq2 定义一个大窗口，seq1 必须在该窗口内某个连续子区间完成匹配。
> within 继承了 intersect 的"匹配长度对齐"复杂性，建议先明确"窗口边界"再写断言。

▶ 示例代码

```
sequence win;

  @(posedge clk) start ##[1:5] ender; // 大窗口：start 到 ender

endsequence


sequence sub;

  @(posedge clk) mid ##1 mid;         // 子序列：连续两拍 mid

endsequence


ap: assert property (@(posedge clk) disable iff(!rst_n) (sub within win))
```

```
    else $error("sub not within window");
```

▶ 注意事项

> 用于"异常必须发生在某个事务期间"，"某个事件必须在 timeout 窗口内出现"。

Q19 【高级】 `strong(sequence_expr)` 与 `weak(sequence_expr)` 有什么区别?

▶ 答案

> • strong(seq)：在且仅在存在非空匹配时为真，要求最终出现一次完整匹配。
> • weak(seq)：在且仅在不存在"有限前缀证明无法匹配"的情况下为真；对无限延迟/未终止场景更宽松。
> 工程理解：strong 更像"必须真的发生一次完整匹配"；weak 更像"只要没被证明不可能即可"。
> 对 timeout/最终性属性更常用 s_eventually 表达；strong/weak 更常用于把 sequence 当 property 的特殊形式。

▶ 示例代码

```
sequence s;

  @(posedge clk) a ##[1:$] b; // b 何时出现不确定

endsequence


// 强：要求最终出现一次非空匹配

ap_strong: assert property (@(posedge clk) disable iff(!rst_n) strong(s))

             else $error("strong(s) failed");


// 弱：在某些工具/语义下可容忍未终止匹配

ap_weak:   assert property (@(posedge clk) disable iff(!rst_n) weak(s))

             else $error("weak(s) failed");
```

▶ 注意事项

> strong/weak 的细节与"有限/无限时间字"和形式语义强相关，关键场景应回到标准条文确认。

Q20 【中级】 如何用 `not/and/or` 组合属性来表达更复杂的规则？请写一个"禁止读写同时发生"的断言，并给出变体。

▶ 答案

> SVA 支持对 property 做逻辑组合（not/and/or 等），便于把多条约束拆成原子条件再组合。
> • not property：取反，"禁止"语义。
> • and：两个属性都必须满足。
> • or：任一属性满足即可。

> 工程技巧：把多个原子 property 组合成更大规则，并分别用 cover 证明每个子属性都被激活。

▶ 示例代码

```
// 并发属性（在每个 clk tick 检查）

property p_not_rw_same_cycle;

  @(posedge clk) disable iff(!rst_n) not (Read && Write);

endproperty

ap: assert property(p_not_rw_same_cycle) else $error("Read/Write conflict");


// 变体：立即断言（贴近组合语义）

always_comb begin

  assert (!(Read && Write)) else $error("comb conflict");

end
```

▶ 注意事项

> 若 Read/Write 是跨时钟域/异步信号，应明确需求是"同步可见性"还是"真实电气瞬态"再选断言类型。

第三章：时序操作符与高级序列操作（Q21–Q32）

Q21 【初级】 `##n` 与 `##[m:n]` 如何工作？`$` 在区间里表示什么？

▶ 答案

- ##n：固定 n 个时钟周期延迟。
- ##[m:n]：在 m 到 n 个时钟周期后的某个 tick 满足后续表达式。
- ##[m:$]：$ 表示无穷上界（eventuality 型表达）。

注意：无限上界会引入"未终止尝试"（pending）与形式证明收敛问题；很多项目用 timeout（有限上界）替代无界 eventual。

▶ 示例代码

```
// 固定：req 后 3 拍 gnt

ap_fixed: assert property (@(posedge clk) disable iff(!rst_n) req |-> ##3 gnt);


// 区间：req 后 1~3 拍 gnt

ap_rng:   assert property (@(posedge clk) disable iff(!rst_n) req |-> ##[1:3] gnt);
```

```
// eventual：req 后某个未来时刻 gnt（可能导致永远等问题，慎用）

ap_evt:   assert property (@(posedge clk) disable iff(!rst_n) req |-> ##[1:$] gnt);
```

▶ 注意事项

把 ##[1:3] 与 first_match 组合可在多匹配情况下挑选最早满足点。

Q22 【中级】 `##0` 表达的是什么？在 `req |-> ##0 ack` 中，`ack` 在哪个采样点被检查？

▶ 答案

##0 表示零个周期延迟的连接：后续元素在同一 clock tick 的匹配点上继续评估（不推进到下一 tick）。

req |-> ##0 ack 等价于 req |-> ack（overlapped implication）：在 req 为真的同一采样 tick 对 ack 做检查。

注意：并发断言仍是 tick 采样模型，周期内短脉冲若在采样点前后消失，可能不会被看到。

▶ 示例代码

```
// 同拍要求 ack 为真（与 req |-> ack 语义完全相同）

// req |-> ##0 ack 等价于 req |-> ack

ap_same_tick: assert property (@(posedge clk) disable iff(!rst_n) req |-> ack)

                else $error("ack must be true in same tick as req");
```

▶ 注意事项

若要捕获电气毛刺，应考虑立即断言或更底层建模，并发断言只在 tick 采样。

⚠ 勘误说明

原文 Q22 代码中使用了 req |-> ##0 ack，写法合法但 ##0 是冗余的，因为 |-> 本身就是 overlapped（同拍）含义。建议直接写 req |-> ack，代码更清晰，语义完全等价。

Q23 【中级】 consecutive repetition `[*]`、`[+]`、`[*m:n]` 的语义是什么？

▶ 答案

- [*N]：操作数序列连续匹配 N 次，例如 a[*3] 等价于 a ##1 a ##1 a。
- [*]：等价于 [*0:$]（零次或多次）。
- [+]：等价于 [*1:$]（至少一次）。
- [*0:3]：允许 0 次匹配（可选项），可能引入空匹配与 vacuity，写 [*1:3] 往往更安全。

常用于：保持类条件（握手信号保持、stall 连续周期）或 burst 长度检查。

▶ 示例代码

```
// b 必须连续为 1 三拍，然后下一拍 c 为 1

ap: assert property (@(posedge clk) disable iff(!rst_n)

      a |-> (b[*3] ##1 c))
```

```
    else $error("repetition failed");
```

▶ **注意事项**

[*0:n] 允许 0 次匹配，相当于可选项，可能引入 vacuity；写 [*1:n] 往往更符合"至少一次"语义。

Q24 【高级】 goto repetition `[->]` 与 nonconsecutive repetition `[=]` 的差异是什么?

▶ **答案**

两者都对布尔表达式进行迭代匹配，在若干个（不必连续）满足点之间匹配。区别：

- [->N]（goto repetition）：整体匹配结束于最后一次迭代匹配点当下。
- [=N]（nonconsecutive repetition）：类似 goto，但允许在最后一次匹配之后延长若干个 tick，匹配终点不必是最后一次满足点当下。

两者只适用于布尔表达式。若需求是"连续 N 次"，应使用 [*N]。

▶ **示例代码**

```
// start 后，b 在区间内出现恰好 2 次（不必连续），然后下一拍 done

ap_goto: assert property (@(posedge clk) disable iff(!rst_n)

            start |-> (b[->2] ##1 done))

          else $error("goto repetition failed");


// 变体：允许最后一次 b 后拉长空窗再到 done

ap_non:  assert property (@(posedge clk) disable iff(!rst_n)

            start |-> (b[=2] ##1 done))

          else $error("nonconsecutive repetition failed");
```

▶ **注意事项**

常用于"事件出现 N 次后必须发生某事"（如 retry 次数、credit 消耗）。

Q25 【中级】 为什么 SVA 特别容易栽在"运算符优先级/结合性"上？给出一个必须加括号的例子。

▶ **答案**

SVA 的 sequence 操作符优先级（从高到低大致为）：[*]/[=]/[->] > ## > throughout > within > intersect > and > or。

缺括号往往不会编译报错，却会产生与意图不同的匹配区间/线程分裂。

工程建议：对所有非平凡序列操作强制括号，并遵循团队 style guide（每条 property 拆行对齐）。

▶ **示例代码**

```
// 正确：明确括号表达 (a ##1 b) or c
```

```
ap_good: assert property (@(posedge clk) disable iff(!rst_n)

            ((a ##1 b) or c))

          else $error("good expr");


// throughout/within/intersect 等操作符与 ## 结合时，不加括号极易出错

// 例: (!burst_mode throughout (a ##1 b)) intersect (c ##1 d)

// 必须加括号明确嵌套关系
```

▶ 注意事项

对 throughout/within/intersect 尤其建议括号，它们把"区间"概念引入组合，误解成本高。

Q26 【高级】 `.triggered` 方法如何用于"序列结束事件化"？标准为什么说 `ended` 已被弃用？

▶ 答案

sequence_instance.triggered 方法在当前 tick 判断序列实例是否到达 end point，从而把"序列完成"当作可引用的条件。
ended 方法与 .triggered 含义相同，但后来被标准标记为 deprecated（已弃用），建议统一使用 .triggered。
.triggered 关注"结束点"，不约束"起始点"；若需精确纳入起始点条件，应直接引用 sequence 本身。

▶ 示例代码

```
sequence s_abc;

  @(posedge clk) a ##1 b ##1 c;

endsequence


// 当 s_abc 完成时（end point），.triggered 为真，然后下一拍 start 必须为 1

property p_after_abc_then_start;

  @(posedge clk) disable iff(!rst_n)

    s_abc.triggered |-> ##1 start;

endproperty


ap: assert property(p_after_abc_then_start) else $error("start not after abc");
```

▶ 注意事项

将复杂 sequence 的 completion 作为后续 property 的 antecedent，降低长序列直接嵌套的复杂度。

Q27 【中级】 `$rose/$fell/$stable/$changed` 的语义是什么？它们比较的是"采样值"还是"实时值"？

▶ 答案

这些函数属于 value change functions，比较的是采样值与"最近一次严格更早的采样值"：

- $rose(expr)：当前采样值为 1，且前一采样值为 0（或 X/Z）。
- $fell(expr)：当前采样值为 0，且前一采样值为 1（或 X/Z）。
- $stable(expr)：当前采样值与前一采样值相同。
- $changed(expr)：当前采样值与前一采样值不同。

注意：信号在 tick 之间变化又回到原值，value change function 仍可能返回"稳定"，因为它只看离散采样点。

▶ 示例代码

```
// req 上升沿后 2 拍 ack 上升沿

ap: assert property (@(posedge clk) disable iff(!rst_n)

      $rose(req) |-> ##2 $rose(ack))

    else $error("req/ack edge protocol broken");


// valid 窗口内 data 必须稳定

ap_stable: assert property (@(posedge clk) disable iff(!rst_n)

            (req && !ack) |-> $stable(data))

          else $error("data changed while waiting");
```

▶ 注意事项

用 $changed 检查"只允许在某些相位变化"；用 $stable 检查"握手等待期间数据保持"。

Q28 【中级】 `$past(expr, n, enable)` 这种写法是什么意思？有哪些合法性限制？

▶ 答案

$past(expression1 [, number_of_ticks] [, expression2] [, clocking_event]) 返回在过去第 n 个采样 tick 的值；提供 expression2 时，采样基于"clock iff expression2"的 gated 采样语义。

例如 $past(d, 2, enable) 表示基于 posedge clk iff enable 的采样历史取两步前值。

限制：某些自动变量在 clocking event 或 $past 的 expression2 中使用是非法的；仿真早期缺乏足够历史，需用 reset/disable 控制或加条件保护。

▶ 示例代码

```
// 当 done 时，q 应等于 enable 为真时采样到的两次更新前值

ap: assert property (@(posedge clk) disable iff(!rst_n)
```

```
      done |=> (q == $past(d, 2, enable)))

    else $error("past-gated check failed");
```

▶ **注意事项**

> 常用于验证寄存器延迟链、pipeline 对齐、或 gating enable 下的有效更新历史。

Q29 【高级】 `nexttime` 与 `s_nexttime` 有何差别？`eventually` 与 `s_eventually` 又有何差别？

▶ **答案**

> 强弱区别（s_ 前缀为强形式）：
> - nexttime p：弱形式，如果没有下一个 tick，该属性为真（不强制存在后续 tick）。
> - s_nexttime p：强形式，要求至少还有一个 tick，且该 tick 上 p 为真。
> - eventually [m:n] p：弱 eventually，不允许无界上界（eventually [2:$] 为非法）。
> - s_eventually p：强 eventually，要求在当前或未来某个 tick 真的满足（允许无界）。
>
> s_eventually 是表达 liveness（最终会发生）的常用操作符。

▶ **示例代码**

```
// 强 eventually：要求未来某个 tick a 为真（允许无界）

ap_se: assert property (@(posedge clk) disable iff(!rst_n) s_eventually a)

        else $error("a never becomes true");


// 区间强 eventually（弱 eventually [2:$] 是非法的，应使用 s_eventually）

ap_se_rng: assert property (@(posedge clk) disable iff(!rst_n) s_eventually[2:$] a)

            else $error("a not eventually true in [2:$]");
```

▶ **注意事项**

> 面试常问：liveness 用 s_eventually，safety 用 not/always；必须同时说明 reset/disable。

Q30 【高级】 `until/s_until` 与 `until_with/s_until_with` 的差别是什么？强弱与重叠/非重叠如何组合？

▶ **答案**

> 四种形式：
> - p1 until p2：弱、非重叠：p1 在 p2 成立之前的 tick 保持为真，不要求 p2 必须发生。
> - p1 s_until p2：强、非重叠：同上，但要求 p2 必须在未来某刻发生。
> - p1 until_with p2：弱、重叠：p1 保持为真，包含 p2 成立的那一 tick。
> - p1 s_until_with p2：强、重叠：同上，且要求 p2 必须发生。
>
> 把"必须最终完成"的需求写成弱 until 会导致"永远 busy 也不报错"。

▶ 示例代码

```
// busy 必须保持为真，直到 done 发生（弱：done 可能永远不发生也不算失败）

ap_weak: assert property (@(posedge clk) disable iff(!rst_n) busy until done);


// 强：done 必须在未来某刻发生，否则失败

ap_str:  assert property (@(posedge clk) disable iff(!rst_n) busy s_until done);
```

▶ 注意事项

用于建模"握手完成前维持某状态"，"reset 释放后保持 init 直到 ready"。

Q31 【高级】 `accept_on/reject_on/sync_accept_on` 与 `disable iff` 的区别是什么?

▶ 答案

- disable iff：作用在 concurrent assertion level，disable 条件为真时 attempt 进入 disabled 状态，不计入 pass/fail。
- accept_on(cond) p：作用在 property level；当 abort 条件（cond）为真时，让 property 评估变为 true（主动接受，视为成功），而非 disable。使用 sampled values。
- reject_on(cond) p：当 abort 条件为真时，强制 property 评估为 false。
- sync_accept_on/sync_reject_on：只在 clocking event 的 time steps 评估 abort 条件。

▶ 示例代码

```
// abort 为真时直接接受（property 评估为 true）

ap_acc: assert property (@(posedge clk) accept_on(abort) (a |-> ##1 b))

        else $error("accept_on: violated");


// reject_on：abort 为真时强制失败

ap_rej: assert property (@(posedge clk) reject_on(abort) (a |-> ##1 b))

        else $error("reject_on: violated");
```

▶ 注意事项

事务级协议中 cancel/flush 信号到来时，用 accept_on(flush) 表达"不追责该事务"。

Q32 【高级】 如何写一个"跨两个时钟域"的 SVA（multi-clock）？多时钟的对齐语义有什么坑?

▶ 答案

SVA 支持在 sequence_expr 中引入不同的 clocking_event，在多时钟 implication 中，antecedent 结束后会等待 consequent 时钟最近的 tick 再开始评估。

最难的坑：若属性没有唯一语义 leading clock，可能直接非法；不同工具对多时钟 pending/mature 的展示可能不同。

CDC 场景：在 clkA 域检测输入事件，在 clkB 域等待输出事件。

▶ 示例代码

```
// 在 A 域看到 A_in 上升后，切到 B 域等待 B_out 在 2~3 个 B 域周期内上升

property p_cdc;

  @(posedge Clk_A) disable iff(!rst_n)

    $rose(A_in) |-> ( @(posedge Clk_B) ##[2:3] $rose(B_out) );

endproperty


ap: assert property (p_cdc) else $error("CDC latency violated");
```

▶ 注意事项

多时钟断言的语义应以标准的 clock resolution 与 semantic leading clock 规则为准。

第四章：采样语义、复位与多时钟（Q33–Q40）

Q33 【高级】 并发断言使用的是"采样值"。clocking block 的 input skew（例如 1step、#0）与 Observed/Reactive 区有什么关系？

▶ 答案

clocking block 的 input 采样时机：

- 默认 input #1step：在前一 time step 结束时采样（最安全，避免 race）。
- input #0（observed deferred）：在当前 time step 的 Observed region 采样，以避免与 DUT NBA 更新的竞态。

cb 变量与直接信号引用的采样点可能不同，是"断言莫名其妙 fail"的常见根因。

在 testbench 中驱动信号时，强烈建议对断言也统一使用 cb 变量。

▶ 示例代码

```
clocking cb @(posedge clk);

  default input #1step; // 在前一 time step 末采样

  input a;

endclocking


// 使用 cb.a：按 clocking block 采样语义取值
```

```
ap_cb: assert property (@(posedge clk) cb.a |-> ##1 cb.a);


// 直接使用 a：按 property 的 clocking event 采样（可能有 race）

ap_a:  assert property (@(posedge clk) a |-> ##1 a);
```

▶ 注意事项

在 program/clocking block 驱动与采样时，统一使用 cb 变量减少 race 是最佳实践。

Q34 【中级】 `default clocking` 与 `default disable iff` 的作用域与覆盖规则是什么？

▶ 答案

default disable iff 规则：

- 在 module/interface/program 或 generate block 作用域内生效，与声明位置无关。
- 同一作用域中多次声明 default disable iff 是错误。
- 其作用域不会扩展到模块实例内部。

覆盖规则：

- 具体断言自身带了 disable iff -> 覆盖默认 disable。
- 要取消默认 disable -> 显式写 disable iff(1'b0)。

▶ 示例代码

```
module q34_default_disable (input logic clk, rst, rst1, a, b);

  default disable iff (rst); // 默认复位条件


  // 明确覆盖默认 disable

  ap1: assert property (@(posedge clk) disable iff (rst1) a |=> b);


  // 使用默认 disable iff(rst)

  ap2: assert property (@(posedge clk) a |=> b);


  // 取消默认 disable

  ap3: assert property (@(posedge clk) disable iff (1'b0) a |=> b);

endmodule
```

▶ 注意事项

陷阱：默认 disable 不会传进子模块实例，只对当前声明作用域与嵌套声明（不是实例）生效。

Q35 【中级】 `disable iff` 的语义是什么？为什么有人会纠结它与 clocking event 的先后位置？

▶ 答案

disable iff(expr) 为并发断言提供异步终止/禁用机制：当 disable 条件成立时，当前 attempt 进入 disabled 状态（不计入 pass/fail 统计），并停止该尝试继续监控。

标准的 property_spec 语法：[clocking_event] [disable iff(...)] property_expr。

注意：disable iff 不是同步 reset，它在 attempt 运行期间任意时刻为真都可能终止 attempt；复位释放边沿附近容易出现被 disable 掉而没检查到的现象。

▶ 示例代码

```
// 推荐写法：clocking event 后跟 disable iff

ap: assert property (@(posedge clk) disable iff(!rst_n) req |-> ##1 ack)

      else $error("disabled-aware check failed");
```

▶ 注意事项

常用 default disable iff(reset) 统一复位；协议取消场景考虑 accept_on 更贴近"取消不算错"语义。

Q36 【中级】 什么是 procedural concurrent assertion？它与 immediate assertion 在语义上有什么关键差别？

▶ 答案

并发断言可以嵌入过程块（如 always @(posedge clk)）：仍然是并发断言，不是立即断言。

区别：

- 模块外静态断言：从仿真开始到结束持续在每个 tick 启动 attempt。
- 过程并发断言：受过程执行路径控制（可能被条件分支跳过），"过程触发时把一个并发断言实例（pending）入队"，随后可能成熟并在后续 tick 被检查。

应用：在状态机分支内只对特定模式启用断言，无需把 enable 写进 property。

▶ 示例代码

```
property rule;

  @(posedge clk) disable iff(!rst_n) a |-> b ##1 c;

endproperty


always @(posedge clk) begin

  // 过程并发断言：到达此语句时，启动一次并发属性评估线程

  assert property (rule) else $error("procedural concurrent failed");

end
```

▶ 注意事项

过程并发断言受过程路径控制，可能被 if/case 分支跳过，这与静态断言持续触发不同。

Q37 【中级】 `bind` 语句的基本语义是什么？如何正确地把一个断言模块绑定到目标模块？

▶ 答案

bind 允许将一个模块/接口实例化到目标 scope（模块或实例）中，把断言与检查逻辑"外部挂载"到设计层次结构里。

- bind <module_name> <sva_module> <inst>(...)：绑定到目标模块的所有实例。
- bind <hier_path> <sva_module> <inst>(...)：精准绑定到指定实例。

bind 语句在同一 scope 中的相对顺序通常不重要，工具可自由选择 elaboration 顺序。

▶ 示例代码

```
module dut2_sva(input logic clk, rst_n, req, ack);

  ap: assert property (@(posedge clk) disable iff(!rst_n) req |-> ##1 ack)

        else $error("bind: req->ack failed");

endmodule


bind dut2 dut2_sva u(.clk(clk), .rst_n(rst_n), .req(req), .ack(ack));
```

▶ 注意事项

bind 模块引用 DUT 内部信号时，最好通过端口连接保证可移植性，避免设计层次变化导致绑定失败。

Q38 【中级】 为什么很多团队喜欢把协议断言写在 interface 里？请给出一个 ready/valid interface + 断言的例子。

▶ 答案

把协议断言写在 interface 的三个工程价值：

1. 信号与断言天然共址（减少端口散落）。
2. 可通过 modport 明确方向与可见性。
3. 对多个模块复用同一套协议检查。

注意："一接口一时钟域"是最佳实践，避免多时钟域复用导致语义混乱。

▶ 示例代码

```
interface rv_if (input logic clk, input logic rst_n);

  logic valid, ready;

  logic [31:0] data;


 // 当 valid=1 且 ready=0, data 必须稳定
```

```
  ap_stable: assert property (@(posedge clk) disable iff(!rst_n)
                (valid && !ready) |-> $stable(data))
              else $error("data changed while stalled");

  // valid 拉高后必须保持到握手完成
  ap_hold_valid: assert property (@(posedge clk) disable iff(!rst_n)
                    $rose(valid) |-> (valid throughout (!ready[*0:$] ##1 ready)))
                  else $error("valid not held until ready");
endinterface
```

▶ 注意事项

对标准总线（APB/AXI/TileLink）将协议断言做成 interface/assertion IP，通过 bind 挂在不同实例上。

Q39 【高级】 checker 是什么？与"断言模块 + bind"相比，它解决了什么工程问题？

▶ 答案

checker 是标准定义的"可封装验证实体"：

- checker body 可包含 let/sequence/property/function、deferred assertions、concurrent assertions、covergroup 甚至嵌套 checker。
- checker 有输出参数，并可继承声明点的 default clocking 与 default disable iff。
- 核心价值：把多条断言 + 覆盖 + 辅助函数/变量 + 参数化封装为可复用单元。

限制：不能在 fork-join 中实例化 checker，也不能在另一个 checker 的 procedure 中实例化。

▶ 示例代码

```
checker q39_checker (logic req, ack, event clock, input logic rst_n, output bit failure);
  default clocking @clock; endclocking
  default disable iff (!rst_n);

  property p_req_ack;
    req |-> ##1 ack;
  endproperty

  a1: assert property (p_req_ack)
        failure = 1'b0;
      else begin
```

```
        failure = 1'b1;
        $error("checker: req->ack failed");
      end

  c1: cover property (req ##1 ack);
endchecker

module q39_checker_use(input logic clk, rst_n, req, ack);
  bit fail;
  q39_checker ck(req, ack, posedge clk, rst_n, fail);
endmodule
```

▶ 注意事项

把协议检查封装为 checker，让 SoC 中多个实例共享；结合 free checker variables 可用于形式验证环境建模。

Q40 【高级】 写 SVA 时有哪些"性能与仿真注意事项"？如何避免线程爆炸、降低调试成本？

▶ 答案

性能热点来源：无界时间窗（##[1:$]/[*0:$]）、可产生大量匹配的 antecedent、并行组合导致的 thread 分裂。

常用策略：

1. 给窗口加上界（用 timeout 代替无界 eventual）。
2. 在 antecedent 上加 gating 或 first_match。
3. 用 disable iff / $assertcontrol 在非关键阶段关闭断言。
4. 通过清晰 label 与分层（bind file/checker/interface）提升定位效率。

ABV 方法论：assert=设计必须满足，assume=环境必须满足，cover=你希望看到的场景。

▶ 示例代码

```
// 不推荐：无界 eventual 可能长期 pending
// ap_bad: assert property (@(posedge clk) req |-> ##[1:$] ack);

// 推荐：加 timeout 上界
ap_timeout: assert property (@(posedge clk) disable iff(!rst_n) req |-> ##[1:16] ack)
             else $error("timeout: ack not seen within 16 cycles");
```

```
// 若 ack 可能多次出现，first_match 限制线程数量

sequence s_ackwin; ##[1:16] ack; endsequence

ap_fm: assert property (@(posedge clk) disable iff(!rst_n) req |-> first_match(s_ackwin))

        else $error("ack window fail");
```

▶ 注意事项

把"环境假设"写成 assert 会导致大量误报；把"设计要求"写成 assume 会掩盖 bug。

第五章：覆盖率与断言可观测性（Q41–Q45）

Q41 【中级】 `cover property` 与 `cover sequence` 在"成功计数/多匹配"上有什么本质差异?

▶ 答案

• cover property：覆盖计数每次 attempt 最多加 1；结果包含 attempted、succeeded 以及"因 vacuity 成功次数"字段。

• cover sequence：同一次 attempt 可能产生多个 matches，统计按 multiplicity 计数。

选择：只关心"是否发生过" -> cover property；关心"发生了多少次/每个 attempt 的多重匹配" -> cover sequence。

▶ 示例代码

```
sequence s;

  @(posedge clk) a ##1 b;

endsequence


property p;

  @(posedge clk) disable iff(!rst_n) a |-> ##1 b;

endproperty


cs: cover sequence ( @(posedge clk) disable iff(!rst_n) s );

cp: cover property ( p );
```

▶ 注意事项

把"想覆盖某个片段出现次数"写成 cover property 可能低估次数，需要真实计数时用 cover sequence。

Q42 【中级】 如何从 coverage 报表判断"断言没失败但也没验证过"？应关注哪些字段?

▶ 答案

属性覆盖结果中的关键字段：

- attempted：启动的 attempt 总数。
- succeeded：成功的 attempt 数（每次最多 +1）。
- succeeded because of vacuity：因 vacuity 成功的次数。
- disabled：被 disable iff 终止的次数（不计入成功统计）。

典型坏味道：attempted 大、succeeded 大、但 vacuity 成功占比极高 -> antecedent 很少成立或断言被 reset/disable 掉。

只看"0 assertion failure"不够；必须结合 cover 证明场景触发过。

▶ 示例代码

```
property p;

  @(posedge clk) disable iff(!rst_n) req |-> ##1 ack;

endproperty

ap: assert property(p);


// 健康度指标：req 是否出现过

c_req: cover property (@(posedge clk) disable iff(!rst_n) req);


// 统计 p 的 vacuity 成功比例

c_p:   cover property(p);
```

▶ 注意事项

在 regression dashboard 上把"assertion pass + cover hit + scoreboard pass"作为三维指标。

Q43 【高级】 如何把 `cover property(...) <pass_statement>` 与 covergroup 结合，实现"命中该 cover 点时采样数据分布"？

▶ 答案

cover statement 带可选 pass statement，在匹配完成的 time step 的 Reactive region 执行。

因此可在 pass statement 中调用 cg.sample()，实现"只有覆盖点命中时才采样 covergroup"。

注意：不要在 cover 的 pass statement 里做过重的工作（大量打印/复杂循环），会拖慢仿真；建议只做采样或计数。

▶ 示例代码

```
covergroup cg @(posedge clk);

  coverpoint data;

endgroup
```

```
cg u_cg = new();


// 只有 req->ack 命中时才采样 data

c_req_ack: cover property (@(posedge clk) disable iff(!rst_n) req |-> ##1 ack)

            u_cg.sample();
```

▶ **注意事项**

对协议事务的 payload 分布做覆盖（只有握手成功时采样 address/data/id）。

Q44 【高级】 在形式验证里，`cover property` 常用于什么？如何用它做 reachability?

▶ **答案**

形式工具对 cover property 做"是否存在一条可达路径满足该行为"的分析，从而用 witness trace 证明场景可达（或证明不可达）。

形式 cover 命中仅意味着存在路径，不代表概率高；若想提升仿真命中率，需要在 stimulus/约束上做引导。

常见 formal session 顺序：1. environment cover -> 2. safety proof -> 3. cover reachability。

▶ **示例代码**

```
// 形式场景：找一条路径使得 start 后 10 拍内出现 done

c_reach: cover property (@(posedge clk) disable iff(!rst_n)

          start ##[1:10] done);
```

▶ **注意事项**

结合 restrict/assume 收敛状态空间后，用 cover 寻找特定 corner case（如 FIFO full 后立即读写交错）。

Q45 【中级】 如何把断言库做成宏（macro）以统一风格？以 OpenTitan 风格的 `COVER(name, prop, clk, rst)` 为例说明。

▶ **答案**

宏化的价值：统一 clock/reset/命名规范，减少团队成员写法差异，并易于全局启停。

宏参数中的 (prop) 必须用括号包起来，避免运算符优先级导致展开后语义变化；宏内的 disable iff 极性要与团队 reset 约定保持一致。

▶ **示例代码**

```
`define ASSERT(name, prop, clk, rst_n) \

  name: assert property (@(posedge clk) disable iff(!(rst_n)) (prop))


`define COVER(name, prop, clk, rst_n) \
```

```
  name: cover  property (@(posedge clk) disable iff(!(rst_n)) (prop))


module q45_macro_lib(input logic clk, rst_n, req, ack);

  `ASSERT(ap_req_ack, (req |-> ##1 ack), clk, rst_n)

  `COVER (cp_req_ack, (req |-> ##1 ack), clk, rst_n)

endmodule
```

▶ **注意事项**

> 进一步扩展宏为"带 severity 与消息"的版本；或加入 $assertcontrol 作用域标记，方便分组启停。

第六章：形式验证与工程化设计题（Q46–Q50）

Q46 【高级】 `assume property` 与 `restrict property` 有何区别？为什么说 restrict "仿真不验证、无 action block"?

▶ **答案**

> • assume property：形式验证中约束输入空间；仿真中也会检查假设是否被违反，有 action block。
> • restrict property：语义与 assume 相同，但专门给形式工具使用：不在仿真中验证（仿真中被忽略），且没有 action block。
> 因此 restrict 是"专门给形式工具吃的约束"，不会在仿真里帮你发现环境违例。
> 若也希望仿真检查环境是否满足约束，应使用 assume property。

▶ **示例代码**

```
// 形式约束：只验证 ctr==0 的场景（仿真中该语句被忽略）

restrict property (@(posedge clk) disable iff(!rst_n) (ctr == 1'b0));


// 若也希望仿真检查，应使用 assume property

// assume property (@(posedge clk) disable iff(!rst_n) (ctr == 1'b0));
```

▶ **注意事项**

> 把 restrict 当仿真约束会导致"仿真里根本没生效"；反之把 assume 当形式约束但不给足环境条件会导致反例不可达。

Q47 【高级】 形式验证最常见三类错误是什么？如何用断言/覆盖避免"证明了但没意义"?

▶ **答案**

> 三类常见陷阱：
> 1. 过约束：把输入空间限制过窄，属性变得"平凡为真"。

2. 欠约束：环境条件缺失，产生不可达 counterexample 或证明依赖于不真实环境。
3. 假设本身有 bug：证明的是"假世界"里的正确性。

SVA 层面对策：

- 用 cover property 验证关键前提与关键场景可达。
- 对环境约束用 assume/restrict 分层，并将"环境合法性"也用 assert/cover 做交叉校验。
- 对 liveness 属性用强形式（s_eventually/s_until），避免弱形式让"永远不发生"也算通过。

▶ 示例代码

```
// 你想证明：start 后最终 done

ap: assert property (@(posedge clk) disable iff(!rst_n) start |-> s_eventually done);


// 自检：start 必须可达，否则 ap 可能长期 vacuous

c_start: cover property (@(posedge clk) disable iff(!rst_n) start);
```

▶ 注意事项

如果 start 在形式模型里永远不可达，主断言可能永远 vacuous success；cover 自检是形式 flow 的健康指标。

Q48 【高级】 设计题：为一个 ready/valid 接口写一组断言（至少包括：valid 保持、数据稳定、最大延迟）。

▶ 答案

规则：

1. valid 拉高表示数据有效，直到握手完成（valid && ready）前必须保持为 1。
2. 当 valid && !ready 时，data 必须保持稳定。
3. valid 拉高后，最多 MAX_LAT 个周期内必须等到 ready（不能无限 stall）。

常见陷阱：复位期间未加 disable iff 导致大量误报；bounded latency 的窗口边界要与 spec 对齐；throughout 约束的是匹配区间，窗口构造错误会导致 hold 约束覆盖范围不对。

▶ 示例代码

```
parameter int MAX_LAT = 8;


// 1) valid 拉高后必须保持到握手完成

ap_hold_valid: assert property (@(posedge clk) disable iff(!rst_n)

    $rose(valid) |-> (valid throughout (!ready[*0:$] ##1 ready)))

  else $error("valid not held until ready");


// 2) 等待期间 data 必须稳定

ap_data_stable: assert property (@(posedge clk) disable iff(!rst_n)
```

```
    (valid && !ready) |-> $stable(data))
  else $error("data changed during stall");

// 3) bounded latency：valid 后 MAX_LAT 周期内必须出现 ready
ap_bounded: assert property (@(posedge clk) disable iff(!rst_n)
    $rose(valid) |-> ##[0:MAX_LAT] (valid && ready))
  else $error("handshake did not complete within MAX_LAT");
```

▶ **注意事项**

> 规则3可改为"ready 可以一直为 0，但 valid 允许撤回（cancel）"，则应考虑 accept_on(cancel) 或将 cancel 纳入 disable 逻辑。

Q49 【高级】 设计题：为一个同步 FIFO 写断言（空/满、非法读写、指针更新一致性、以及功能覆盖）。

▶ **答案**

规则：

1. empty 时禁止 rd_en；full 时禁止 wr_en。
2. full 与 empty 不应同时为 1。
3. 写后计数 +1，读后计数 -1（假设已暴露 count）。
4. 覆盖：至少出现过"写至 full""读至 empty"。

注意：同时读写时 count 不变；count 断言用 |=> 检查下一拍值（寄存器延迟）。

▶ **示例代码**

```
// 1) 非法读写
ap_no_rd_empty: assert property (@(posedge clk) disable iff(!rst_n)
    empty |-> !rd_en) else $error("read when empty");

ap_no_wr_full:  assert property (@(posedge clk) disable iff(!rst_n)
    full  |-> !wr_en) else $error("write when full");

// 2) full/empty 互斥
ap_mutex: assert property (@(posedge clk) disable iff(!rst_n)
    !(full && empty)) else $error("full & empty both 1");

// 3) count 变化（写无读 +1，读无写 -1）
```

```
ap_cnt_inc: assert property (@(posedge clk) disable iff(!rst_n)

    (wr_en && !rd_en && !full) |=> (count == $past(count) + 1))

  else $error("count not incremented");


ap_cnt_dec: assert property (@(posedge clk) disable iff(!rst_n)

    (rd_en && !wr_en && !empty) |=> (count == $past(count) - 1))

  else $error("count not decremented");


// 4) 覆盖点：到 full / 到 empty

c_to_full:  cover property (@(posedge clk) disable iff(!rst_n) full);

c_to_empty: cover property (@(posedge clk) disable iff(!rst_n) empty);
```

▶ 注意事项

> 把 count 替换为读写指针（gray code）时，可用 $past 与 XOR/gray 转换检查指针单 bit 变化，配合 CDC 断言。

Q50 【高级】 Debug 题：为什么断言失败但你看波形觉得"信号是对的"？请从"采样点/事件区/clocking block"角度分析并给出修复方案。

▶ 答案

断言 @(posedge clk) req |-> ##1 ack 波形看起来正常但偶发失败，最常见根因是采样与调度竞态：

- ack 在 posedge 由 DUT 的 NBA 更新，而断言采样/评估与 Observed/Reactive 区间不对齐，可能在某些 race 下看到"旧值"。
- 使用 clocking block 变量（cb.ack）与直接信号引用在采样上可能不同。
- testbench 同时驱动/采样 req/ack 而没有 program/clocking block 规避竞态，导致"波形肉眼看对、断言采样看错"。

修复方案：

1. 引入 clocking block，把采样点固定到 Observed region（input #0）或 1step。
2. 在 property 中加 disable iff(!rst_n)，避免起始不稳定窗口误报。

▶ 示例代码

```
// 修复1：引入 clocking block，把采样点固定

clocking cb @(posedge clk);

  default input #0; // #0 skew：在 Observed region 采样以避免 race

  input req, ack;

endclocking
```

```
ap_cb: assert property (@(posedge clk) disable iff(!rst_n) cb.req |-> ##1 cb.ack);


// 修复2：加 disable iff 避免复位窗口的不稳定导致误报

ap_rst: assert property (@(posedge clk) disable iff(!rst_n) req |-> ##1 ack);
```

▶ 注意事项

> 把断言移动到 program 并通过 clocking block 访问设计信号，是消除 testbench race 的经典方案。
